Kurs Inhalt

- Tag 1:
  - Klassen und Objekte
  - Ableitungen
  - DATETIME-Modul
  - With-Kontextmanager
  - Exceptions
- Tag 2:
  - Iteratoren
  - Higher-Order-Functions
  - Generatoren
  - Dekoratoren
  - Regular Expressions
  - Type Annotations
- Tag 3:
  - DocString
  - Requests
  - Datenbank
  - Datenanalyse Pandas
  - Logging
  - Struktur großer Projekte
  - Unittest
  - UV

Iterator

- In Python

Warum Iteratoren?

```
def get_all_logs():
  all_entries = db.query("SELECT * FROM logs")  # 1 Milliarde Zeilen
  return all_entries

# RAM-Problem: Versucht, 1 Milliarde Objekte *sofort* in den Speicher zu laden.
logs = get_all_logs()
for log in logs:
  # Verarbeitung beginnt erst, NACHDEM alles geladen ist
  process(log)
```

Das Iterator-Protokoll

- Python löst dies durch "Lazy Evaluation" (verzögerte Auswertung).
- Ein Iterator ist ein Objekt, das zwei Methoden implementiert:
  1. `__iter__()`
     - Muss den Iterator selbst zurückgeben (`return self`).
     - Wird (implizit) von `for`-Schleifen aufgerufen.
  2. `__next__()`
     - Gibt das **nächste** Element der Sequenz zurück.
     - Wenn keine Elemente mehr vorhanden sind, löst es die `StopIteration` Exception aus.

iter() & next()

- Python bietet zwei Built-in-Funktionen, um manuell mit dem Protokoll zu arbeiten:

```
my_list = [10, 20, 30]

# 1. iter() holt den Iterator von einem Iterable
my_iterator = iter(my_list)

print(type(my_list))     # <class 'list'>
print(type(my_iterator)) # <class 'list_iterator'>

# 2. next() ruft __next__ auf, um das nächste Element zu holen
print(next(my_iterator)) # Output: 10
print(next(my_iterator)) # Output: 20
print(next(my_iterator)) # Output: 30

# 3. Das Ende der Sequenz
print(next(my_iterator)) # Löst StopIteration aus
```

Iterable vs. Iterator

- `Iterable`
  - Ein Objekt, von dem man einen Iterator **bekommen** kann.
  - Implementiert nur `__iter__()`.
  - Beispiele: `list, tuple, dict, set, str`.
  - Man kann immer wieder `iter()` darauf aufrufen und erhält einen **neuen** Iterator (beginnt von vorn).
- `Iterator`
  - Das Objekt, das die Arbeit macht (das Zählen/Abrufen).
  - Implementiert `__iter__()` **und** `__next__()`.
  - Ist "Stateful" und erschöpft sich (Exhaustible).
  - Ein Iterator ist **auch** ein Iterable (da er `__iter__` hat), aber nicht umgekehrt!

Iterable vs. Iterator

```
# list ist ein Iterable
my_list = [1, 2]

# iterator_1 ist ein Iterator
iterator_1 = iter(my_list)
print(next(iterator_1)) # 1

# iterator_2 ist ein *neuer*, unabhängiger Iterator
iterator_2 = iter(my_list)
print(next(iterator_2)) # 1 (beginnt von vorn)

print(next(iterator_1)) # 2 (iterator_1 war bei Position 1)
```

Lazy Evaluation

- Iteratoren laden Daten „on demand“ – erst wenn `next()` aufgerufen wird.

```
# Datei-Handler in Python sind Iteratoren
  with open("massive_log_file.log", "r") as f:
    # 'f' ist ein Iterator.
    # 'f.readline()' (oder __next__) wird bei jedem Schleifendurchlauf aufgerufen.

    for line in f:
      # Es ist immer nur EINE Zeile gleichzeitig im Speicher!
      process(line)

  # Ergebnis: Verarbeitung einer 100 GB Datei mit < 1 MB RAM.
```

Lab 5

Iteratoren

Higher-Order Functions

- lambda, map, filter, reduce

Wiederholung: Lambda

```
lambda arguments: expression
```

„Wegwerf-Funktionen“ für einfache, einmalige Operationen.

- **Anonym**: Sie haben keinen Namen (kein `def`).
- **Beschränkt**: Nur **eine** Anweisung, kein `return` (implizit).
- **Nützlich**: Perfekt als Argumente für Higher-Order Functions (wie `map`, `filter`, `sorted`).

Wiederholung: Lambda

```
# Traditionelle Funktion
def double(x):
    return x * 2

# Lambda-Äquivalent
double_lambda = lambda x: x * 2

# Nutzung
print(double_lambda(5)) # Output: 10
```

Higher-Order Functions

Python	Java	C#
map	forEach (Java 8)	LINQ (ForEach)
filter	filter (Java 8)	LINQ (where)
reduce	reduce	LINQ (Aggregate)

map(function, iterable)

- Wendet eine Funktion auf **jedes** Element eines Iterables an.
- 1:1 Transformation:
  [x, y, z] → [f(x), f(y), f(z)]

```
numbers = [1, 2, 3, 4]

# Nutzung mit lambda
doubled_iterator = map(lambda x: x * 2, numbers)

# WICHTIG: map() ist "lazy"
print(doubled_iterator)
# Output: <map object at 0x...>

# Um das Ergebnis zu materialisieren:
doubled_list = list(doubled_iterator)
print(doubled_list)
# Output: [2, 4, 6, 8]
```

filter(function, iterable)

- Filtert Elemente aus einem Iterable, basierend auf einer Bedingung.
- N:M Selektion: Die Funktion muss `True` oder `False` zurückgeben. Nur Elemente, für die sie `True` zurückgibt, bleiben erhalten.

```
numbers = [1, 2, 3, 4, 5, 6]

# Die lambda-Funktion ist das Prädikat (Bedingung)
even_iterator = filter(lambda x: x % 2 == 0, numbers)

# filter() ist ebenfalls "lazy"
print(even_iterator)
# Output: <filter object at 0x...>

even_list = list(even_iterator)
print(even_list)
# Output: [2, 4, 6]
```

reduce(function, iterable)

- Reduziert (aggregiert) ein `Iterable` auf einen einzigen Wert durch wiederholte Anwendung einer Funktion.

```
from functools import reduce

numbers = [1, 2, 3, 4]

# 1. (acc=1, el=2) -> 3
# 2. (acc=3, el=3) -> 6
# 3. (acc=6, el=4) -> 10
total = reduce(lambda acc, el: acc + el, numbers)

print(total)
# Output: 10
```

Lab 6

Map, Filter & Reduce

Generatoren

- mit Python

Generatoren: Funktionen

Funktion mit den folgenden Eigenschaften:

- Statement yield statt return
- Automatische Bildung eines Iterator-Objektes, das den internen Zustand über mehrfache Aufrufe der Funktion hält
- Automatische Implementierung einer Methode next() und einer Exception StopIteration
- Vorteil: die Gesamtheit der Daten muß nicht im Speicher sein sondern nur das aktuelle Element: z,B. Lesen einer Datei

Generatoren: Funktionen

```
# Generator Funktion
def csv_reader(file_name):
  for row in open(file_name, 'r'):
    yield row.split(';')


# Verwendung der Funktion
for line in csv_reader('textfile.txt'):
  print(line)
```

Generatoren: Expressions

Expressions mit den folgenden Eigenschaften:

- Automatische Bildung eines Iterator-Objektes, das den internen Zustand hält
- Zusätzliche logische Operationen und Schachtelungen möglich, die übersichtlicher als eine Generator Funktion sind
- Automatische Implementierung einer Methode next() und einer Exception StopIteration
- Vorteil: die Gesamtheit der Daten muß nicht im Speicher sein sondern nur das aktuelle Element: z,B. Lesen einer Datei

Generatoren: Funktionen

```
# Generator Expression
genexpr = (<EXPRESSION> for <ITEM> in <COLLECTION> if <CONDITION>)


# Entspricht der Generator Funktion
def generator():
    for item in collection:
        if condition:
            yield expression
```

Generatoren: Funktionen

```
# Generator-Expression
genexpr = (expr for x in xs if cond1 ... for z in zs if condN)


# Entspricht der Generator-Funktion
def generator():
  for x in xs:
    if cond1:
      ...
      for z in zs:
        if condN:
          yield expr
```

Lab 7

Generatoren

Decorator

- mit Python

Decorator

- Decorator erweitert die Funktionalität von bestehenden Funktionen, ohne diese Funktionen zu verändern
- Decorator Funktion ist ein Wrapper, der eine Vor- und Nachverarbeitung bei Funktionsaufrufen machen kann.
- Decorator Klasse ist ein Wrapper für Funktionen, die einen internen Zustand als Klassenattribute verwalten kann (Zusätzlich zu der Vor- und Nachverarbeitung)
- Anwendungen:
  - Logging/Debugging
  - Zeitmessung
  - Webframework (Verarbeitung der Headers und Payloads)
  - Permissions abfragen
  - Caching

Decorator

```
# Wrapper: Übergabe ist eine Funktion
def rahmen(func):
  # Sorgt für einen Namen der Funktion
  @functools.wraps(func)
  # Innere Funktion
  def wrapper_do_action(*args, **kwargs):
    print('Vorher')
    ret = func(*args, **kwargs)
    print('Nachher')
    return ret

  # Return der Wertes der Funktion
  return wrapper_do_action
```

Lab 8

Dekoratoren

Regular Expressions

- mit Python

Regular Expressions

- Mit Hilfe einer einfachen Suche ohne regular Expressions können nur Teilstrings oder einfache Wildcards gesucht werden. Regular Expressions erlauben eine komplexere Suche in Zeichenketten (z.B alle Wörter, die einen definierten Start und ein definiertes Ende haben. Der Teil dazwischen ist variabel.
- Man definiert durch die Verwendung von Steuerzeichen, Schachtelungen und Wiederholungen eine Grammatik (allgemeine Regeln für den Aufbau der Zeichenkette).

Regular Expressions

Zeichenklasse	Bedeutung
\d	Passt auf alle Ziffern des Dezimalsystems. Ist äquivalent zu [0-9].
\D	Passt auf alle Zeichen, die nicht Ziffern des Dezimalsystems sind. Ist äquivalent zu [^0-9].
\s	Passt auf alle Whitespace-Zeichen. Ist äquivalent zu [\t\n\r\f\v].
\S	Passt auf alle Zeichen, die kein Whitespace sind. Ist äquivalent zu [^ \t\n\r\f\v].
\w	Passt auf alle alphanumerischen Zeichen und den Unterstrich. Ist äquivalent zu [a-zA-Z0-9_].
\W	Passt auf alle Zeichen, die nicht alphanumerisch und kein Unterstrich sind. Ist äquivalent zu [^a-zA-Z0-9_].

Regular Expressions

Sonderzeichen	Bedeutung
\A	Passt nur am Anfang eines Strings.
\b	Passt nur am Anfang oder Ende eines Wortes. Ein Wort kann aus allen Zeichen der Klasse \w bestehen und wird durch ein Zeichen der Klasse \s begrenzt.
\B	Passt nur, wenn es sich nicht um den Anfang oder das Ende eines Wortes handelt.
\Z	Passt nur am Ende eines Strings.
^	Passt nur am Anfang eines Strings. Wenn das MULTILINE-Flag gesetzt wurde, passt ^ auch direkt nach jedem Newline-Zeichen innerhalb des Strings.*
$	Passt nur am Ende eines Strings. Wenn das MULTILINE-Flag gesetzt wurde, passt $ auch direkt vor jedem Newline-Zeichen innerhalb des Strings.*

Regular Expressions

Quantor	Bedeutung
?	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf entweder keinmal oder einmal vorkommen.
*	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf beliebig oft hintereinander vorkommen, das heißt unter anderem, dass sie auch weggelassen werden kann.
+	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf beliebig oft hintereinander vorkommen, mindestens aber einmal. Sie darf also nicht weggelassen werden.

Regular Expressions

Wiederholungen einer Zeichengruppe

Syntax	Bedeutung
{anz}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss exakt anz-mal vorkommen.
{min,}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss mindestens min-mal vorkommen.
{,max}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse darf maximal max-mal vorkommen.
{min,max}	Das vorangegangene Zeichen bzw. die vorangegangene Zeichenklasse muss mindestens min-mal und darf maximal max-mal vorkommen.

Regular Expressions

Wiederholungen einer Zeichengruppe

Syntax	Bedeutung
()	Gruppierung (Zusammenfassung von Zeichen zu einer Einheit. z.B. (Python)+ => „Python“ kann ein oder mehrmals vorkommen
|	Alternativen z.B. P(ython|eter) => „Python“ oder „Peter“
(?i)	Extension (Flag): case insensitive
(?-i)	Extension (Flag): case sensitive

Regular Expressions

```
import re

print(re.findall(r'\bk[a-z]*', 'drei kleine katzen'))
# => Ausgabe ['kleine', 'katzen‘]

print(re.findall(r'\b(k[a-z]*n\b)', 'drei kleine katzen'))
# => Ausgabe ['katzen‘], Anfang und Ende wird gesucht

print(re.findall(r'\b(A\d{3}\w{1}\d{1})', 'A012B1 A01B1'))
# => Ausgabe ['A012B1‘], Anzahl wird überprüft

print(re.findall(r'\b(A\d{3}[BC]\d{1})', 'A012B1 A01B1 A456C2'))
# => Ausgabe ['A012B1', 'A456C2‘]

print(re.findall(r'\b(A\d{3}(DE|FG)\d{1})', 'A012DE1 A01B1 A456FG2'))
# => Ausgabe [('A012DE1', 'DE'), ('A456FG2', 'FG')]
```

Lab 9

Regular Expressions